Java 18 - 21

Java 21 New Features

1. **Virtual Threads (JEP 444)**
    a. This is the "crown jewel" of Java 21. Traditionally, Java threads were "Platform Threads" mapped 1:1 to OS threads, which are expensive and limited in number.
    b. **What they are:** Lightweight threads that allow you to run millions of concurrent tasks on a single JVM.
    c. **Why it matters:** It makes the "thread-per-request" model for web servers (like Spring or Micronaut) incredibly scalable without the complexity of reactive programming.
2. **Sequenced Collections (JEP 431)**
    a. Before Java 21, there was no uniform way to access the first or last element of different collection types (e.g., `List` vs `LinkedHashSet`).
    b. **New Interfaces:** `SequencedCollection`, `SequencedSet`, and `SequencedMap`.
    c. **New Methods:** `getFirst()`, `getLast()`, `addFirst()`, `addLast()`, and `reversed()`.
3. **Pattern Matching for switch (JEP 441)**
    a. You can now use a switch to check the type of an object and deconstruct it in one go.
    b. **Feature**: It supports "Guards" (using when) to add extra conditions.

```
static String formatter(Object obj) {
    return switch (obj) {
        case Integer i -> String.format("int %d", i);
        case String s  -> String.format("string %s", s);
        case Point(int x, int y) when x > 0 -> "Point in positive X";
        default -> obj.toString();
    };
}
```

4. **Record Patterns (JEP 440)**
   a. Records were introduced in Java 16, but Java 21 allows you to **deconstruct** them directly in `instanceof` or `switch` checks.
   b.
```
if (obj instanceof Point(int x, int y)) {
    System.out.println(x + y); // No need to call point.x() or point.y()
}
```

1. Scope Values
   How Scoped Values Work
   A Scoped Value is **immutable** and has a **defined lifecycle**. Once it is bound, it cannot be changed, and once the execution "scope" ends, the data is automatically discarded.

```
public final static ScopedValue<String> USER_ID = ScopedValue.newInstance();

// Inside some method:
ScopedValue.where(USER_ID, "admin123").run(() -> {
    // Everything called inside this block can access USER_ID
    System.out.println(USER_ID.get());
    performAction();
});
// Outside the block, USER_ID is no longer accessible/bound.
```

Scoped Values vs. ThreadLocal: A Quick Comparison

Feature	ThreadLocal	ScopedValue
Mutability	Mutable (set())	**Immutable**
Lifetime	Until thread dies or manual remove	**Limited to a code block**
Memory	High (potential leaks)	**Low (auto-cleanup)**
Virtual Thread Friendly	No (too heavy)	**Yes (highly optimized)**

Why do we need an alternative to `ThreadLocal`?

While `ThreadLocal` has been around since Java 1.2, it has three major flaws that become problematic in the era of Virtual Threads:

2. **Unconstrained Mutability:** Any code can call `ThreadLocal.set()` at any time. This makes it hard to track where data is changing.
3. **Unbounded Lifetime:** Data often stays in a `ThreadLocal` until the thread dies. Because we now use millions of Virtual Threads or reuse Platform Threads in pools, this leads to **memory leaks**.
4. **Expensive Inheritance:** Inheriting `ThreadLocal` data from a parent thread to a child thread requires copying the entire map, which is slow and memory-intensive.

Key Benefits

1. **Security & Predictability**
   Since Scoped Values are **immutable**, you don't have to worry about a library or a downstream method accidentally changing your security context or transaction ID.
2. **Extreme Efficiency**
   In Virtual Threads, Scoped Values are optimized so that child threads can "inherit" the value from the parent thread at **zero cost**. Instead of copying a map, the child thread simply points back to the parent's scope.
3. **Automatic Cleanup**
   There is no need to call `.remove()`. As soon as the `run()` or `call()` method finishes, the binding is gone. This eliminates the risk of "dirty" data leaking from one web request to the next.